# Compiler Verification

Scott Brinley

# Dave, M. A. (2003). Compiler Verification: A Bibliography

- Survey paper listing 100 different papers on compiler verification dating back to 1967
- From this list found the papers that would form the beginning of my presentation
- Survey doesn't provide any analysis or comparison between papers

# Pnueli, A., Siegel, M., & Singerman, E. (1998). Translation validation

- Verifying a compiler prevents future iteration without re-verification
- Rather than directly verifying the compiler, instead checks if the compiled code is a correct representation of the original code using a semantic framework known as STS
- Applied the framework to a compilation from SIGNAL to sequential C code
- Their implementation was limited to just compilations from SIGNAL to C

# Necula, G. C. (2000). Translation validation for an optimizing compiler

- Translation validation previously limited for optimizing compilers
- Uses a two step inference algorithm to determine if the symbolic states are identical
- Evaluated on real world C code bases using GCC, demonstrating its flexibility at the cost of a 4x overhead on compile time
- False alarms certain hard to detect optimizations, such as loop unrolling

## Yang, X., Chen, Y., Eide, E., & Regehr, J. (2011). Finding and understanding bugs in C compilers

- Realistically hard to verify large compilers, existing testing methods are insufficient
- Uses static and dynamic analysis to generate random test cases without undefined behavior, allowing for differential testing
- Over 3 years, uncovered more than 325 unknown bugs across 11 C compilers
- Lacks support for many common language features, doesn't guarantee termination

# Wang, X., Lazar, D., Zeldovich, N., Chlipala, A., & Tatlock, Z. (2014). Jitk: A trustworthy in-kernel interpreter infrastructure

- Operating systems run interpreters with kernel privileges, meaning a bug in one of them could crash the entire OS. This is especially prominent with “just in time” compilers
- Built an infrastructure that guarantees functional correctness, termination, and bounded stack usage
- Integrated into Linux kernel, with similar code size and minor overhead
- Assumes correctness of unproven tools, such as the Coq proof checker. Does not guarantee protection against JIT spraying, which could be subject for future work

# Tate, R., Stepp, M., Tatlock, Z., & Lerner, S. (2009). Equality saturation: a new approach to optimization

- Compilers have a phase-ordering problem, meaning one optimization may prevent another
- Uses PEGs and EPEGs to represent all possible optimizations as functional expressions
- Uses heuristic to approximate best solution given an EPEG

# Program Expression-Graphs (PEG)

- Referentially transparent, meaning the expressions have no side-effects
- Complete, meaning no need to store control flow graphs (CFGs)
- Merged into E-PEGs to prevent exponential space complexity

# PEGs (cont)

- Each node in a PEG represents an operation, children of that node represent inputs, shown by solid (non-dashed) lines
- θ nodes used to represent values that vary within a loop - left is initial, right is iterative value
- ϕ nodes function as multiplexers

# EPEG Generation

- Apply equality axioms to PEGs to generate EPEGs (more sophisticated trigger/callback mechanism used for more complex optimizations)
- Existing nodes are reused rather than recreated, saving significant space
- After saturation, global profitability heuristic picks the best optimization

# PEGs and EPEGs Example

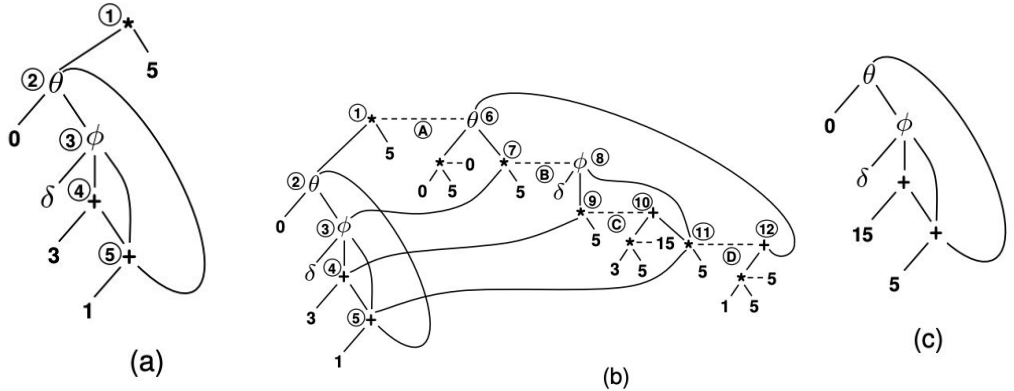
```
i := 0;                          i := 0;
while (...) {                    while (...) {
   use(i * 5);                      use(i);
   i := i + 1;                      i := i + 5;
   if (...) {                       if (...) {
      i := i + 3;                      i := i + 15;
   }                                }
}                                }
          (a)                              (b)
```

**Figure 1.** Loop-induction-variable strength reduction: (a) shows the original code, and (b) shows the optimized code.

$$(a+b)*m = a*m + b*m \quad (1)$$

$$\theta(a,b)*m = \theta(a*m, b*m) \quad (2)$$

$$\phi(a,b,c)*m = \phi(a, b*m, c*m) \quad (3)$$



# Evaluation

- Used Peggy, a Java bytecode optimizer and evaluated against SpecJVM benchmark
- Saturated 84% of methods without requiring bounds
- Used as a translation validator for the Soot compiler, exposing a miscompilation bug

# Limitations and Future Work

- Certain programs never reach saturation (e.g loop unrolling a while true loop)
- Pseudo-Boolean solver to find best set of optimizations was very slow on certain graphs and the bottleneck of the author's equality saturation implementation

The following table shows the average time in milliseconds taken per method for the 4 main Peggy phases (for Pueblo, a timeout counts as 60 seconds).

| | CFG to PEG | Saturation | Pueblo | PEG to CFG |
|---|---|---|---|---|
| Time | 13.9 ms | 87.4 ms | 1,499 ms | 52.8 ms |

- Extending the equality saturation engine to a machine-checkable proof of equivalence

## Willsey, M., Nandi, C., Wang, Y. R., Flatt, O., Tatlock, Z., & Panchekha, P. (2021). egg: Fast and extensible equality saturation

- Equality saturation, while powerful, suffered from severe performance bottlenecks
- Implement a technique called “rebuilding” that drastically improves the asymptotic complexity of building EPEGs
- Aggregate performance of rebuilding was 87.85x faster while equality saturation was only 20.96x faster
- Extracting minimal proof certificates is NP-Hard, but has been solved by later research

# Conclusion: Takeaways

- It can often be easier to verify if a solution is correct instead of proving that you will always generate a correct solution
- Fundamental programming can be incredibly helpful for compact representations of equivalent states
- Heavy formal verification is best used for the most performance critical applications

# Conclusion: What's next?

- Translation validation for AI
- eBPF is the standard for Linux - formally verifying eBPF is still an active point of research
- Verifying that a compiler doesn't break lock-free correctness by reordering variable accesses
- Scaling translation validation to work for more significant compiler optimizations